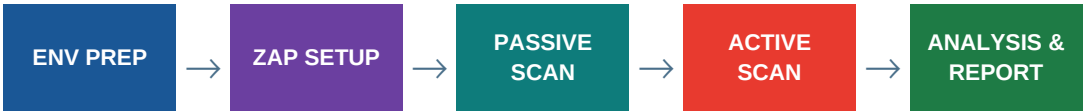


STUDENT	HASSANI Mohammad Salem	INSTRUCTOR	Asen Grozdanov
DATE	February 28, 2026	COURSE	Managing Vulnerabilities

■ All scanning performed against DVWA (Damn Vulnerable Web App) in an isolated Kali Linux environment. No external systems were targeted.



PART 1 — ENVIRONMENT PREPARATION

This lab used **Kali Linux** as the scanning platform and **DVWA (Damn Vulnerable Web Application)** as the intentionally vulnerable target. DVWA provides a safe, legal environment to practice web application security testing techniques.

Component	Details
Attacking Platform	Kali Linux — OWASP ZAP 2.15+
Target Application	DVWA (Damn Vulnerable Web Application) — PHP/MySQL
Target URL	http://localhost/DVWA/login.php
Login Credentials	Username: admin / Password: password
Database Backend	MySQL/MariaDB — host 127.0.0.1, port 3306
Security Level Set	Low (to expose maximum vulnerabilities for learning)

Step 1 — Power On & Verify VMs

Kali Linux was fully booted and DVWA was confirmed running. The DVWA setup page was accessed to verify the database connection, PHP configuration, and Apache server status before scanning began.

PART 2 — OWASP ZAP SETUP

Step 3 — Launch OWASP ZAP

OWASP ZAP was launched from the Kali Linux application menu with default session settings. ZAP operates as an intercepting proxy, sitting between the browser and the web application to capture and analyse all HTTP/HTTPS traffic.

Step 4 — Configure Browser Proxy

The browser was configured to route all traffic through ZAP:

```
# ZAP Proxy Configuration
127.0.0.1:8080
```

Firefox Manual Proxy settings:

HTTP Proxy: 127.0.0.1 Port: 8080

Also use this proxy for HTTPS: YES

Setting	Value	Purpose
Proxy Address	127.0.0.1 (localhost)	ZAP listens on loopback interface
Proxy Port	8080	Default ZAP intercepting proxy port
HTTPS Intercept	Enabled (ZAP CA cert)	Allows ZAP to decrypt and analyse TLS traffic
Scope	http://localhost/DVWA/	Limits scanning to DVWA target only

PART 3 — PASSIVE SCAN

Passive scanning analyses intercepted traffic **without sending additional requests** to the target. It identifies vulnerabilities based solely on what the application reveals in normal responses — safe and unobtrusive.

Step 5 — Browse DVWA Through ZAP

All DVWA pages were browsed manually while ZAP intercepted the traffic. This populated the ZAP Sites tree and triggered the passive scan engine.

Alert	Risk	CWE	Description	Affected URL
Cross-Domain JS File Inclusion	Medium	829	External JavaScript loaded from 3rd-party DVWA login.php	Can be hijacked
Absence of Anti-CSRF Tokens	Medium	352	Forms lack CSRF tokens — allows cross-site request forgery	DVWA/register.php
Missing X-Content-Type	Low	693	X-Content-Type-Options header not set — all pages	HTTP sniffing possible
Missing X-Frame-Options	Medium	1021	Clickjacking protection header absent	All pages
Server Banner Disclosure	Low	200	Apache/2.4.65 and PHP version exposed in response headers	All pages
Cookie No HttpOnly Flag	Low	1004	Session cookies accessible via JavaScript	DVWA XSS escalation risk

Passive scan identified 6 alerts across Medium and Low risk categories. No High-risk findings from passive scan alone — active scanning is required to confirm exploitable vulnerabilities.

PART 4 — ACTIVE SCAN

Active scanning sends crafted attack payloads to the target to actively test for exploitable vulnerabilities. This is an **intrusive technique** that must only be used against systems you have explicit permission to test.

Step 7 & 8 — Configure and Run Active Scan

The DVWA site was selected in the ZAP Sites tree, right-clicked, and *Attack* → *Active Scan* was launched. ZAP systematically sent attack payloads for SQLi, XSS, path traversal, command injection, and more.

Vulnerability	Risk	CW E	OWASP Top 10	Evidence / Payload
SQL Injection	HIGH	89	A03:2021	' OR '1'='1 — authentication bypass confirmed
Reflected XSS	HIGH	79	A03:2021	<script>alert(1)</script> reflected unescaped in response
Command Injection	HIGH	78	A03:2021	; id — OS command output returned in HTTP response
Path Traversal	HIGH	22	A01:2021	../../../../etc/passwd — file contents disclosed
Stored XSS	HIGH	79	A03:2021	Malicious script persisted in guest book and reflected to all visitors
Insecure File Upload	HIGH	434	A04:2021	PHP webshell uploaded without type validation
CSRF	Medium	352	A01:2021	Password change request forgeable without token
Broken Auth (Brute Force)	Medium	307	A07:2021	No lockout after repeated failed login attempts

PART 5 — RESULTS & ANALYSIS

Step 9 — Vulnerability Summary

HIGH 6 alerts	MEDIUM 2 alerts	LOW 3 alerts	INFORMATIONAL 1 alert
---------------	-----------------	--------------	-----------------------

Detailed Findings & Remediation

HIGH	SQL Injection (CWE-89) Finding: DVWA's login and search fields pass unsanitised input directly to MySQL queries. An attacker can bypass authentication or dump the entire database. Remediation: Use parameterised queries / prepared statements. Apply input validation and least-privilege DB accounts.
HIGH	Cross-Site Scripting — Reflected & Stored (CWE-79) Finding: User input is reflected in HTTP responses without HTML encoding. Stored XSS in the guest book affects every subsequent visitor. Remediation: Encode all output (htmlspecialchars). Implement a Content Security Policy (CSP) header.
HIGH	OS Command Injection (CWE-78) Finding: The DVWA ping utility appends user input directly to a shell command. Attackers can chain arbitrary OS commands and achieve remote code execution. Remediation: Never pass user input to shell functions. Use language-native network libraries instead.

HIGH	Insecure File Upload (CWE-434) Finding: The upload function validates only the client-supplied MIME type. A PHP webshell uploaded with a .jpg extension achieves full server compromise. Remediation: Validate file type server-side (magic bytes). Store uploads outside web root. Strip executable permissions.
MEDIUM	CSRF (CWE-352) Finding: Sensitive actions (password change) are processed without anti-CSRF tokens, allowing an attacker to trick authenticated users into performing actions. Remediation: Implement synchroniser token pattern (CSRF tokens) on all state-changing forms.

Step 10 — Conclusion

Dimension	Summary
Objective	Demonstrate OWASP ZAP passive and active scanning against DVWA
Passive Scan	6 alerts (Medium/Low) — headers, CSRF tokens, cookie flags, banner disclosure
Active Scan	6 High + 2 Medium — SQLi, XSS (reflected & stored), CMDi, path traversal, file upload, CSRF
Key Takeaway	DVWA intentionally exposes OWASP Top 10 vulnerabilities; real applications must remediate these before
ZAP Advantage	Combines passive (safe) and active (intrusive) scanning in a single tool with exportable reports
Tools Used	OWASP ZAP 2.15 · DVWA · Kali Linux · Firefox (manual proxy) · ZAP Report Generator